

HTTP'nin DETAYLARI

[Kaynak](#)

İÇERİK

1. HTTP ve HTTPS Temelleri
2. HTTP İstekleri ve Yanıtları (Requests & Responses)
3. HTTP Metotları (HTTP Methods)
4. HTTP Durum Kodları (Status Codes)
5. HTTP Başlıkları (Headers)
6. Çerezler (Cookies)

HTTP ve HTTPS Temelleri

İnternetin bel kemiği olan bu protokolleri anlamak, web tabanlı saldırı vektörlerini (Web Hacking) kavramak için ilk şarttır. Bu notlarda, verinin bir sunucudan tarayıcımıza nasıl geldiğini ve güvenliğin bu sürecin neresinde devreye girdiğini detaylandıracağız.

HTTP Nedir? (HyperText Transfer Protocol)

HTTP, 1989-1991 yılları arasında Tim Berners-Lee ve ekibi tarafından geliştirilen, web sunucuları ile iletişim kurmak için kullanılan standart kurallar dizisidir. Temel amacı, web sayfası verilerinin (HTML dosyaları, görseller, videolar vb.) aktarılmasını sağlamaktır.

Bir web sitesine girdiğimizde tarayıcımız (client), sunucuya bir **HTTP Request** (İstek) gönderir; sunucu ise buna karşılık **HTTP Response** (Yanıt) döner. Bu protokol olmasaydı, farklı cihazların ve tarayıcıların birbirleriyle veri alışverişi yapması imkansız olurdu çünkü ortak bir "dil" konuşuluyor olmazdı.

HTTP'nin Temel Özellikleri:

- **Stateless (Durumsuz) Yapı:** HTTP genellikle "stateless" bir protokoldür. Yani sunucu, bir önceki isteği hatırlamaz. Her istek birbirinden bağımsızdır (Bu yüzden oturum yönetimi için **Cookies** ve **Sessions** gibi ek yapılar kullanılır).
- **Plain Text (Açık Metin):** HTTP üzerinden iletilen tüm veriler açık metin halindedir. Eğer aradaki ağ trafiğini bir araçla (örneğin **Wireshark**) dinlerseniz, kullanıcı adlarını, şifreleri ve tüm içerikleri hiçbir engel olmadan okuyabilirsiniz.

HTTPS Nedir? (HyperText Transfer Protocol Secure)

HTTPS, HTTP protokolünün güvenli hale getirilmiş versiyonudur. "S" harfi **Secure** (Güvenli) anlamına gelir. Teknik olarak HTTP verilerinin **TLS/SSL** (Transport Layer Security / Secure Sockets Layer) protokolü ile sarmalanarak şifrelenmesiyle oluşur.

HTTPS Neden Hayatidir?

HTTPS sadece "şifreleme" demek değildir; siber güvenlik perspektifinden iki ana sütunu vardır:

- Gizlilik (Privacy/Encryption):** Veriler gönderilmeden önce şifrelenir. Bu sayede, "Man-in-the-Middle" (Ortak Adam) saldırısı yapan bir saldırgan trafiği yakalasa bile, elinde sadece anlamsız karakter dizileri olur. Gönderdiğiniz kredi kartı bilgisi veya aldığınız özel bir mesaj başkaları tarafından okunamaz.
- Kimlik Doğrulama (Authentication):** HTTPS, konuştuğunuz sunucunun gerçekten iddia ettiği kişi olduğunu garanti eder. Örneğin, `bankam.com` adresine gittiğinizde, tarayıcınız sunucunun **Dijital Sertifikasını** kontrol eder. Eğer sertifika geçerli değilse, birinin sizi sahte bir sunucuya (impersonation) yönlendirdiğini anlarsınız.

Özet Karşılaştırma

Özellik	HTTP	HTTPS
Port	Genellikle 80 portunu kullanır.	Genellikle 443 portunu kullanır.
Güvenlik	Şifreleme yoktur, veriler kabak gibi ortadadır.	TLS/SSL ile uçtan uca şifreleme sağlar.
Sertifika	İhtiyaç duymaz.	SSL Sertifikası zorunludur.

Kritik Not: Penetrasyon testlerinde bir hedef incelenirken ilk bakılması gereken şeylerden biri sitenin HTTPS kullanıp kullanmadığıdır. Eğer bir login sayfası HTTP üzerinden çalışıyorsa, bu direkt olarak ciddi bir zafiyettir çünkü kimlik bilgileri ağda açıkça izlenebilir.

HTTP İstekleri ve Yanıtları (Requests & Responses)

Web dünyasında her etkileşim bir "İstek-Yanıt" döngüsüdür. Tarayıcımız bir varlık (HTML, görsel, dosya vb.) istediğinde sunucu buna bir yanıt döner. Ancak bu sürecin doğru işlemesi için önce tarayıcıya bu kaynağa **nereden** ve **nasıl** ulaşacağını söylememiz gerekir. İşte burada devreye **URL** giriyor.

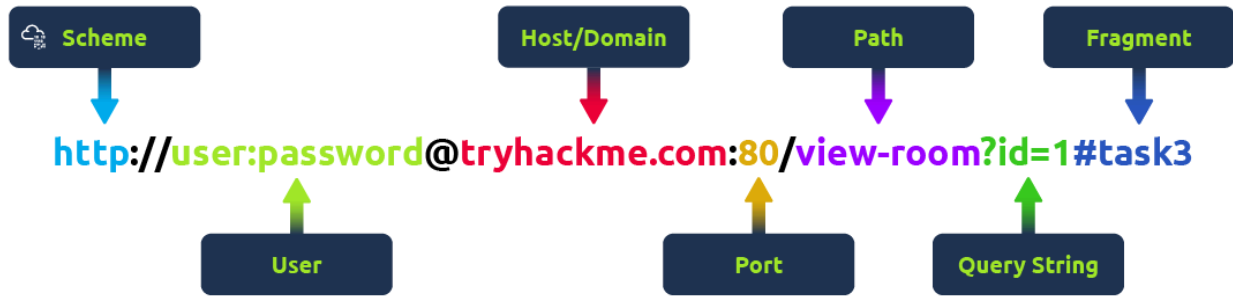
URL Nedir? (Uniform Resource Locator)

URL, internet üzerindeki bir kaynağa nasıl erişileceğini belirten bir talimatlar bütünüdür. Her gün kullandığımız bu adresler aslında karmaşık bir yapıya sahiptir.

URL Bileşenleri

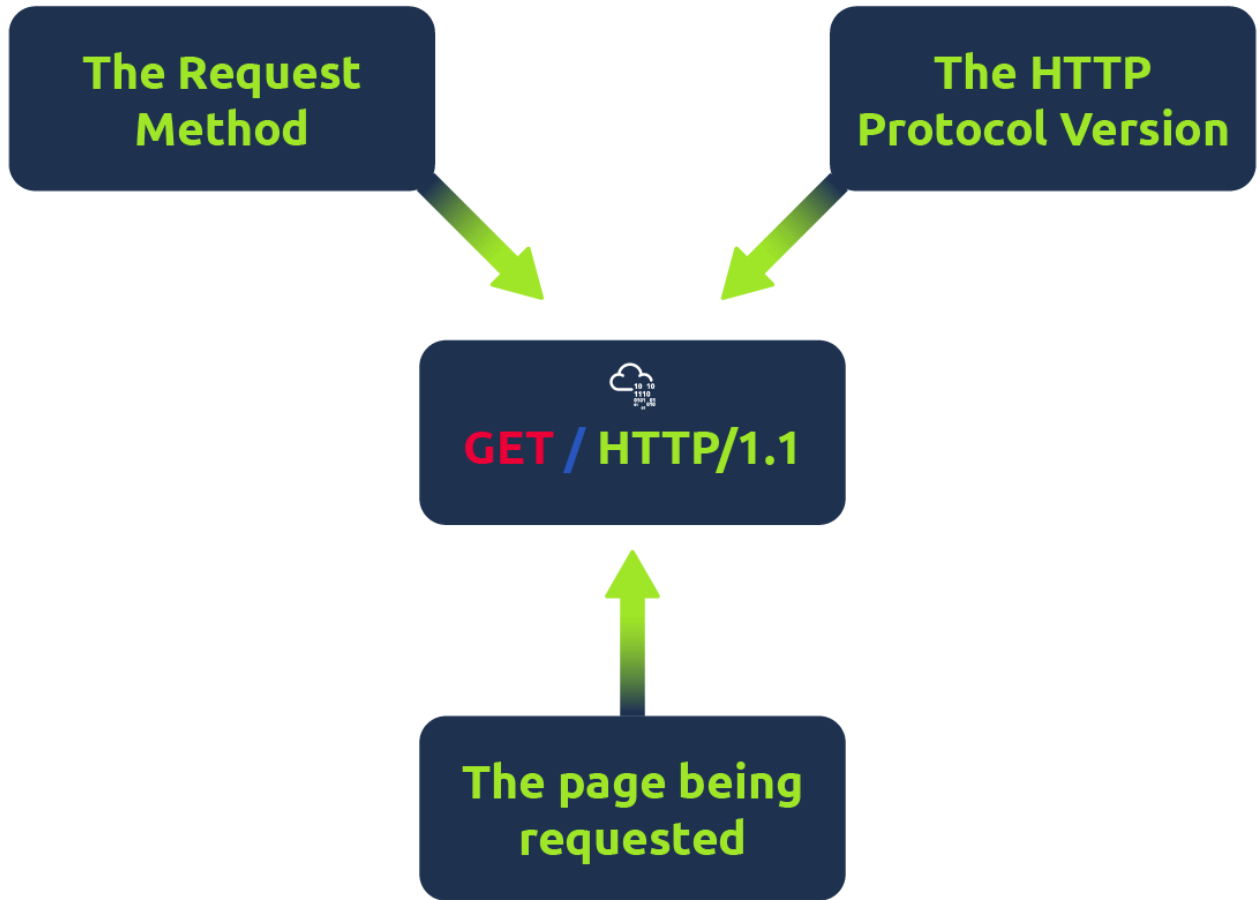
Bir URL'nin anatomisini şu parçalar oluşturur:

1. **Scheme (Şema):** Kaynağa erişmek için hangi protokolün kullanılacağını belirtir. Yaygın olanlar: **HTTP, HTTPS, FTP**.
2. **User (Kullanıcı):** Bazı servisler giriş yapmak için kimlik doğrulaması ister. Kullanıcı adı ve şifre doğrudan URL içine (`kullanıcı:şifre@host`) şeklinde yazılabilir (Güvenlik nedeniyle modern web'de pek tercih edilmez).
3. **Host (Sunucu):** Erişmek istediğiniz sunucunun **Domain adı** (tryhackme.com) veya **IP adresi**.
4. **Port:** Bağlanılacak kapı numarasını belirtir. Varsayılan olarak HTTP için **80**, HTTPS için **443** kullanılır. Ancak bir servis **1 ile 65535** arasındaki herhangi bir portta çalışıyor olabilir.
5. **Path (Yol):** Sunucu üzerinde erişilmek istenen dosyanın veya kaynağın tam konumu.
6. **Query String (Sorgu Dizisi):** İstenen yola gönderilen ek parametrelerdir. Soru işareti (`?`) ile başlar.
 - Örnek: `/blog?id=1` ifadesinde sunucuya "blog sayfasını getir ama ID'si 1 olan makaleyi ver" demiş oluruz.
7. **Fragment (Fragman):** Sayfa içindeki belirli bir konuma atıfta bulunur. Kare (`#`) işareti ile kullanılır. Sayfa yüklendiğinde tarayıcının doğrudan o bölüme (örneğin bir alt başlığa) kaymasını sağlar.



HTTP İsteği Yapmak (Making a Request)

Bir web sunucusuna en basit haliyle tek satırlık bir istek bile atılabilir: `GET / HTTP/1.1`. Ancak modern web için sunucuya daha fazla detay vermemiz gerekir. Bu detaylar **Headers** (Başlıklar) kısmında iletilir.



Örnek Bir İstek Analizi:

HTTP

```
GET / HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Referer: https://tryhackme.com/
```

- **1. Satır:** GET metodu (veri almak istediğimizi belirtir), / kök dizini (ana sayfa) ve kullanılan protokol versiyonu olan HTTP/1.1 bilgisini içerir.
- **2. Satır (Host):** Hangi alan adına istek attığımızı netleştirir. Bir IP adresinde birden fazla web sitesi barınabileceği için bu başlık kritiktir.
- **3. Satır (User-Agent):** Sunucuya hangi tarayıcıyı ve işletim sistemini kullandığımızı söyler. Sunucular bazen tarayıcıya göre farklı içerik (mobil vs. masaüstü) dönebilir.
- **4. Satır (Referer):** Sunucuya bu sayfaya hangi adresten yönlendirilerek geldiğimizi söyler. (Analiz ve güvenlik takibi için kullanılır).
- **5. Satır:** HTTP protokolünde isteklerin bittiğini belirtmek için mutlaka bir **boş satır** bırakılır.

HTTP Yanıtı Almak (Example Response)

Sunucu isteđimizi aldıktan sonra bize bir paket gönderir. Bu paketin içinde hem teknik bilgiler hem de asıl içerik yer alır.

Örnek Bir Yanıt Analizi:

HTTP

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.15.8
```

```
Date: Fri, 09 Apr 2021 13:34:03 GMT
```

```
Content-Type: text/html
```

```
Content-Length: 98
```

```
<html>
```

```
<head>
```

```
  <title>TryHackMe</title>
```

```
</head>
```

```
<body>
```

```
  Welcome To TryHackMe.com
```

```
</body>
```

```
</html>
```

- **1. Satır (Status Line):** Kullanılan protokol (HTTP/1.1) ve **Status Code** (Durum Kodu) yer alır. 200 OK her şeyin yolunda olduğunu ve isteđin başarıyla tamamlandığını gösterir.
- **2. Satır (Server):** Sunucuda çalışan web sunucusu yazılımını ve versiyonunu (nginx/1.15.8) belirtir. **Siber güvenlikte bu bilgi çok kritiktir;** saldırganlar bu versiyondaki bilinen açıkları (CVE) araştırabilir.
- **3. Satır (Date):** Sunucunun o anki tarih, saat ve zaman dilimi bilgisi.
- **4. Satır (Content-Type):** İstemciye (tarayıcıya) ne tür bir veri gönderildiđini söyler. text/html geliyorsa tarayıcı bunu bir web sayfası olarak render eder; image/jpeg geliyorsa resim olarak gösterir.
- **5. Satır (Content-Length):** Yanıtın gövdesinin (body) bayt cinsinden uzunluđudur. Bu sayede tarayıcı tüm verinin eksiksiz gelip gelmediđini kontrol eder.
- **6. Satır:** Yine yanıtın başlıklar kısmının bittiđini gösteren **boş satır**.
- **7. Satır ve Sonrası:** Talep edilen asıl veri (HTML kodları).

Önemli Not: Bir siber güvenlikçi için **User-Agent** ve **Server** başlıkları altın değerindedir. Kendi tarayıcınızın kimliđini gizlemek (User-Agent Spoofing) veya hedef sunucunun hangi yazılımı kullandığını anlamak (Banner Grabbing) için bu başlıkları manipüle etmeyi veya okumayı bilmelisiniz.

HTTP Metotları (HTTP Methods)

HTTP metotları, bir istemcinin (tarayıcı veya script) sunucuya gönderdiği istekte **asıl amacının ne olduğunu** belirten "fiillerdir". Sunucu, gelen isteğin başındaki metoda bakarak hangi işlemi yapması gerektiğini anlar. Birçok HTTP metodu olsa da, bir siber güvenlikçi veya geliştirici olarak en çok karşılaşacağın metotlar şunlardır:

1. GET Metodu (Bilgi Alma)

En yaygın kullanılan metottur. Sunucudan belirli bir kaynağı (dosya, sayfa, resim) **getirmek/okumak** için kullanılır.

- **Kullanım Amacı:** Sadece veri okumak. Sunucu üzerinde herhangi bir kalıcı değişiklik yapması beklenmez (Read-only).
- **Veri İletimi:** Veriler URL içindeki **Query String** (sorgu dizisi) kısmında taşınır.
 - **Örnek:** `search.php?sorgu=siber+guvenlik`
- **Güvenlik Notu:** Veriler URL'de açıkça görüldüğü için şifre veya hassas bilgiler GET ile gönderilmemelidir. Bu veriler tarayıcı geçmişine ve sunucu loglarına (kayıtlarına) girer.

2. POST Metodu (Veri Gönderme)

Sunucuya veri **göndermek**, yeni bir kayıt oluşturmak veya bir işlemi başlatmak için kullanılır.

- **Kullanım Amacı:** Form doldurma, dosya yükleme veya giriş yapma (login) işlemleri.
- **Veri İletimi:** Veriler URL'de değil, HTTP isteğinin **Body (Gövde)** kısmında taşınır. Bu sayede çok daha büyük miktarda veri gönderilebilir.
- **Güvenlik Notu:** Veriler URL'de gözükmez ancak bu tek başına güvenli olduğu anlamına gelmez. Eğer HTTPS kullanılmıyorsa, paket yakalandığında gövde içindeki veriler (şifreler dahil) okunabilir.

3. PUT Metodu (Veri Güncelleme)

Sunucudaki mevcut bir kaynağı **güncellemek** veya o kaynak yoksa oluşturmak için kullanılır.

- **Kullanım Amacı:** Mevcut bir profil bilgisini güncellemek veya bir dosyanın içeriğini değiştirmek.
- **POST'tan Farkı:** Genellikle "Idempotent" (aynı işlemi defalarca yapsan da sonuç değişmez) olması beklenir. POST yeni bir kayıt oluştururken, PUT var olanı tam hedef adresiyle günceller.

4. DELETE Metodu (Veri Silme)

Adından da anlaşılacağı üzere, sunucu üzerindeki belirli bir kaynağı **silmek** için kullanılır.

- **Kullanım Amacı:** Bir kullanıcı hesabını silmek, bir makaleyi kaldırmak vb.
- **Siber Güvenlik Perspektifi:** Eğer bir web sunucusunda DELETE metodu yanlış yapılandırılmışsa ve yetkilendirme kontrolü zayıfsa, saldırganlar sunucudaki kritik dosyaları bu metodu kullanarak silebilirler.

Özet Karşılaştırma Tablosu

Metot	İşlem Türü (CRUD)	Veri Nerede?	Sunucuda Değişiklik?
GET	Read (Okuma)	URL (Query String)	Hayır
POST	Create (Oluşturma)	Request Body	Evet
PUT	Update (Güncelleme)	Request Body	Evet
DELETE	Delete (Silme)	URL veya Body	Evet

Not Notu: Bir web sızma testinde (Pentest), hedef sunucunun hangi metotları desteklediğini anlamak için `OPTIONS` metodu kullanılır. Eğer sunucu gereksiz yere `PUT` veya `DELETE` metotlarına izin veriyorsa, bu potansiyel bir saldırı yüzeyidir.

HTTP Durum Kodları (Status Codes)

Bir HTTP sunucusu isteğimize yanıt verdiğinde, yanıtın ilk satırında her zaman bir **durum kodu** bulunur. Bu kod, istemciye (tarayıcıya) isteğin sonucunu ve gerekirse bu sonuçla nasıl başa çıkacağını bildiren dijital bir "cevap anahtarıdır".

Durum Kodu Aralıkları (Sınıflandırma)

HTTP durum kodları, anlamlarını kolayca kavrayabilmemiz için 5 ana kategoriye ayrılmıştır. Kodun ilk rakamı, yanıtın genel doğasını belirler:

- 100-199: Bilgilendirme Yanıtları (Informational):** Sunucunun isteğin ilk kısmını aldığını ve istemcinin geri kalanını göndermeye devam edebileceğini belirtir. Modern web'de artık çok sık karşılaşılmaz.
- 200-299: Başarı (Success):** İsteğin başarıyla alındığını, anlaşıldığını ve kabul edildiğini gösterir. En sevdiğimiz aralıktır.
- 300-399: Yönlendirme (Redirection):** İstenen kaynağın başka bir yere taşındığını belirtir. Tarayıcıyı otomatik olarak yeni bir adrese yönlendirmek için kullanılır.
- 400-499: İstemci Hataları (Client Errors):** Hatanın kullanıcıdan (istemciden) kaynaklandığını belirtir. Yanlış URL, eksik parametre veya yetkisiz erişim denemesi gibi durumlar bu aralıktadır.
- 500-599: Sunucu Hataları (Server Errors):** İstemcinin isteği geçerli olabilir ancak sunucunun bu isteği yerine getirirken bir sorun yaşadığını (çökme, veritabanı hatası, aşırı yüklenme) belirtir.

En Sık Karşılaşılan HTTP Durum Kodları

Sızma testlerinde ve web geliştirmede ezbere bilinmesi gereken kritik kodlar şunlardır:

Başarı Kodları

- **200 OK:** İstek başarıyla tamamlandı. Sunucu istenen veriyi döndürdü.
- **201 Created:** İstek başarılı oldu ve sunucuda yeni bir kaynak oluşturuldu (Örn: Yeni bir kullanıcı kaydı veya blog yazısı).

Yönlendirme Kodları

- **301 Moved Permanently:** Kaynak kalıcı olarak yeni bir URL'ye taşındı. Arama motorları bu kodu görünce eski linki listeden çıkarıp yenisini ekler.
- **302 Found:** Geçici bir yönlendirmedir. Kaynak şu an farklı bir adreste ama ileride eski yerine dönebilir.

İstemci Hatası Kodları

- **400 Bad Request:** Sunucu, isteği hatalı veya eksik olduğu için anlayamadı. Gerekli bir parametreyi göndermediğimizde sıkça alırız.
- **401 Unauthorized:** Bu kaynağı görmek için önce kimlik doğrulaması (giriş) yapmanız gerekiyor. Genellikle kullanıcı adı/şifre hatalarında veya eksikliğinde görülür.
- **403 Forbidden:** Kimliğiniz doğrulanmış olsa bile bu kaynağa erişim yetkiniz yok. (Örn: Bir kullanıcının admin panelini açmaya çalışması).
- **404 Page Not Found:** İstenen sayfa veya dosya sunucuda bulunamadı. URL hatalı olabilir veya dosya silinmiş olabilir.
- **405 Method Not Allowed:** Kullanılan HTTP metodu bu kaynak için desteklenmiyor. Örneğin, sadece POST ile veri kabul eden bir sayfaya GET isteği atarsanız bu hatayı alırsınız.

Sunucu Hatası Kodları

- **500 Internal Server Error:** Sunucu tarafında genel bir hata oluştu. Kod hatası veya yapılandırma bozukluğu olabilir. Hata detayları genellikle güvenlik nedeniyle bu mesajda gösterilmez.
- **503 Service Unavailable:** Sunucu şu an isteği işleyemiyor. Genellikle aşırı trafikten (DDoS saldırısı gibi) veya bakım çalışmasından kaynaklanır.

Siber Güvenlik Notları (Pentester Bakış Açısı)

- **Enumeration (Keşif):** Bir saldırgan, web sitesindeki gizli dizinleri bulmak için araçlar (gobuster, ffuf) kullanırken **404** (yok) ve **403** (yasak ama var) kodlarını takip eder.
- **401 vs 403:** Bir sızma testinde **401** almak, "bir giriş kapısı var, sadece anahtarın yok" demektir; **403** almak ise "kapı kapalı ve içeri girmen kesinlikle yasak" demektir.
- **Hata Mesajları:** Bazen bir **500** hatasıyla birlikte sunucu, kullanılan programlama dili veya veritabanı hakkında teknik detaylar sızdırabilir. Bu bilgiler saldırganın işini kolaylaştırır (Information Disclosure).

HTTP Başlıkları (Headers)

HTTP başlıkları, bir istek (Request) veya yanıt (Response) gönderilirken ana verinin yanına eklenen **ekstra bilgi parçacıklarıdır**. Teknik olarak bir başlık gönderilmeden de HTTP isteği yapılabilir, ancak modern bir web sitesini düzgün görüntülemek veya güvenli bir iletişim kurmak için başlıklar olmazsa olmazdır.

Başlıkları iki ana kategoride inceleyeceğiz: İstemciden gidenler (**Request**) ve sunucudan dönenler (**Response**).

Yaygın İstek Başlıkları (Common Request Headers)

Bu başlıklar tarayıcımızdan (Client) sunucuya (Server) gönderilir. Sunucuya kendimiz ve ne beklediğimiz hakkında bilgi veririz.

- **Host:** Bir IP adresinde (sunucuda) birden fazla web sitesi barınıyor olabilir (Sanal Host yapısı). Bu başlık, sunucuya tam olarak hangi alan adını (örneğin: `tryhackme.com`) istediğimizi söyler. Eğer bu başlık yanlırsa sunucu sizi varsayılan (default) sitesine yönlendirir.
- **User-Agent:** Kullandığınız tarayıcının adı ve versiyon bilgisidir. Sunucu bu bilgiyi kullanarak web sitesini tarayıcınıza uygun şekilde formatlar. Bazı gelişmiş CSS veya JavaScript özellikleri her tarayıcıda çalışmadığı için sunucu bu başlığa göre farklı içerik sunabilir.
- **Content-Length:** Sunucuya veri gönderirken (örneğin bir form doldururken), gönderilen verinin bayt cinsinden büyüklüğünü belirtir. Sunucu bu sayede verinin tamamının ulaşmış olup olmadığını kontrol eder.
- **Accept-Encoding:** Tarayıcınızın hangi sıkıştırma yöntemlerini (gzip, deflate, br vb.) desteklediğini söyler. Böylece sunucu, veriyi sıkıştırarak gönderir ve internet trafiğinden tasarruf edilir.
- **Cookie (Çerez):** Sunucu tarafından daha önce size verilmiş olan verileri sunucuya geri gönderir. Sunucunun sizi tanımasını ve oturumunuzun (session) açık kalmasını sağlar.

Yaygın Yanıt Başlıkları (Common Response Headers)

İsteğimize karşılık sunucunun bize gönderdiği, tarayıcımızın nasıl davranması gerektiğini söyleyen bilgilerdir.

- **Set-Cookie:** Sunucunun tarayıcımıza "Bu bilgiyi sakla ve bundan sonraki her istekte bana geri gönder" dediği başlıktır. Oturum yönetimi için kritiktir.
- **Cache-Control:** Tarayıcının bu yanıtın içeriğini ne kadar süreyle hafızasında (Cache) tutacağını belirler. Bu sayede aynı sayfaya tekrar girdiğinizde her şeyi baştan indirmeniz gerekmez, sayfa daha hızlı açılır.
- **Content-Type:** Tarayıcıya gelen verinin türünü söyler (`text/html` , `application/json` , `image/png` vb.). Tarayıcı bu başlığa bakarak veriyi ekrana mı basacağını yoksa bir dosya olarak mı indireceğini anlar.
- **Content-Encoding:** Sunucunun veriyi gönderirken hangi sıkıştırma yöntemini kullandığını belirtir. Tarayıcı bu bilgiyi kullanarak veriyi açar (decompress).

Siber Güvenlik ve Analiz Perspektifi

Bir siber güvenlik uzmanı için başlıkları okumak ve manipüle etmek temel bir yetenektir:

- Güvenlik Başlıkları:** Bu listede olmayan ancak çok kritik olan `X-Frame-Options`, `Content-Security-Policy` (CSP) veya `Strict-Transport-Security` (HSTS) gibi başlıklar vardır. Bunların eksikliği web sitesini saldırılara açık hale getirir.
- Bilgi Sızdırma (Information Disclosure):** Sunucunun döndüğü `Server` veya `X-Powered-By` gibi başlıklar, sunucuda kullanılan yazılımları (PHP versiyonu, Apache/Nginx versiyonu) sızdırabilir.
- Hileli İstekler (Spoofing):** Bir saldırgan, `User-Agent` başlığını değiştirerek kendini bir arama motoru botu (Googlebot) gibi tanıtabilir veya `Referer` başlığını değiştirerek belirli bir adresten geliyormuş gibi yapabilir.

Önemli Hatırlatma: `Content-Length` başlığı ile `Content-Type` başlığı arasındaki farka dikkat! Biri verinin **boyutunu**, diğeri verinin **türünü** söyler.

Çerezler (Cookies)

Web dünyasının en kritik bileşenlerinden biri olan **Cookies (Çerezler)**, bilgisayarınızda saklanan küçük veri parçacıklarıdır. Önceki derslerde öğrendiğimiz gibi HTTP protokolü **stateless** (durumsuz) bir yapıdadır; yani sunucu sizinle saniyeler önce yaptığı konuşmayı anında unutur. Çerezler, bu hafıza kaybını önleyen ve sunucunun bizi "hatırlamasını" sağlayan dijital kimlik kartlarıdır.

Çerezler Nasıl Çalışır?

Çerezlerin yaşam döngüsü basit bir döngüden ibarettir:

- Set-Cookie (Sunucudan İstemciye):** Bir web sitesine girdiğinizde veya giriş yaptığınızda, sunucu size bir yanıt gönderir. Bu yanıtın başlıkları arasında `Set-Cookie` satırı bulunur. Bu satır tarayıcınıza şunu der: *"Hey, şu veriyi cebine koy ve sakla."*
- Cookie (İstemciden Sunucuya):** Tarayıcınız bu veriyi aldıktan sonra, aynı sunucuya yapacağınız **her bir sonraki istekte** bu veriyi `Cookie` başlığı altında otomatik olarak geri gönderir.

Çerezlerin Kullanım Amaçları

- Oturum Yönetimi (Authentication):** En yaygın kullanımdır. Giriş yaptığınızda sunucu size bir **Token** (eşsiz, tahmin edilemez bir kod) verir. Bu sayede her sayfaya tıkladığınızda tekrar kullanıcı adı ve şifre girmeniz gerekmez.
- Kişiselleştirme:** Gece modu tercihi, dil seçimi veya "Sepetimdeki ürünler" gibi bilgiler çerezlerde tutulabilir.
- Takip (Tracking):** Reklam ağları, siteye daha önce gelip gelmediğinizi anlamak için çerezleri kullanır.

Önemli Güvenlik Notu: Çerezler genellikle şifrenizi açık metin olarak içermez. Bunun yerine **Session Token** denilen karmaşık diziler içerir. Eğer bir saldırgan sizin çerezinizi çalarsa (**Session Hijacking**), şifrenizi bilmesine gerek kalmadan doğrudan hesabınıza giriş yapmış sayılır.

Çerezleri Görüntüleme ve Analiz Etme

Bir siber güvenlik öğrencisi olarak çerezleri nasıl inceleyeceğinizi bilmeniz şarttır. Tarayıcınızın **Geliştirici Araçları (Developer Tools)** bu iş için ana aracımızdır.

Adım Adım İnceleme:

- Geliştirici Araçlarını Açın:** (Genellikle F12 veya Sağ Tık -> İncele / Inspect).
- Network (Ağ) Sekmesine Gidin:** Bu sekme, tarayıcınızın o an yaptığı tüm HTTP trafiğini listeler.
- Bir İsteği Seçin:** Listelenen dosyalardan (HTML, JS veya resim fark etmez) birine tıklayın.
- Headers veya Cookies Sekmesini Kontrol Edin:**
 - Headers** sekmesinde **Cookie** (giden) ve **Set-Cookie** (gelen) başlıklarını görebilirsiniz.
 - Cookies** sekmesi ise (tarayıcıya göre değişir) bu isteğe özel olarak gönderilen tüm çerezleri daha düzenli bir tablo halinde sunar.

Çerezlerin Anatomisi (Örnek)

Bir HTTP isteğinde çerez şöyle görünür:

HTTP

```
GET /dashboard HTTP/1.1
Host: banka.com
Cookie: session_id=abc123xyz789; theme=dark
```

Bu örnekte sunucuya iki bilgi gidiyor:

- session_id** : Sunucunun bizi tanıması için gereken gizli kod.
- theme** : Bizim tercih ettiğimiz karanlık mod ayarı.

Bir Üst Dönem Notu: Çerez Bayrakları (Security Flags)

Çerezlerin güvenliğini artırmak için kullanılan bazı ek parametreler vardır, bunları görmek seni 100'e götürür:

- HttpOnly**: Bu bayrak varsa, çereze JavaScript ile (örneğin bir XSS saldırısıyla) ulaşılamaz. Sadece HTTP isteklerinde gönderilir.

- **Secure:** Çerezin sadece güvenli (HTTPS) bağlantılar üzerinden gönderilmesini zorunlu kılar.
- **SameSite:** Çerezin başka sitelerden gelen isteklerle (CSRF saldırısı gibi) gönderilip gönderilmeyeceğini belirler.